

4-bit Binary Coded Hexadecimal to 7-Segment Display Decoder on Spartan-7 FPGA

A Project Report

Submitted as part of the Internship Program

at

Academy of Skill Development



National Institute of Technology Mizoram



Submitted by
Mohammad Rehan



Certificate from the Mentor

This is to certify that **Mohammad Rehan** has completed the project titled **4-bit Binary Coded Hexadecimal to 7-Segment Display Decoder on Spartan-7 FPGA** under my supervision during the period from **May** to **July** as *part of the Internship Program at Academy of Skill Development.*

Electronics and Communication Engineering of National Institute of Technology Mizoram.

Leblanu Mondal

Signature of the Mentor

Date: 22 JULY 2025

Acknowledgment

I take this opportunity to express my deep gratitude and sincerest thanks to my project mentor, **Mr. Debtanu Mondal** for giving the most valuable suggestions, helpful guidance, and encouragement in the execution of this project work.

I would like to give a special mention to my colleagues. Last but not least I am grateful to all the faculty members of the **Academy of Skill Development** for their support.

Abstract

This project presents a Verilog-based design of a **4-bit binary-coded hexadecimal decoder displayed on a 7-segment display using a Spartan-7 FPGA board**. The input(sw) is taken via onboard slide switches, and the corresponding hexadecimal digit (0–F) is shown on the selected 7-segment display through a display selector(an). The system includes a reset function and supports real-time testing using pushbuttons. This decoder is useful in digital systems for translating binary input into human-readable format via 7-segment output. Simulation and synthesis are performed using Xilinx Vivado, and constraints are set according to the Spartan-7 pin configuration. The results are validated through simulation waveforms and verified on hardware by observing the display outputs. This project demonstrates hardware-level digital logic implementation, FPGA configuration, and efficient use of Verilog for real-time embedded applications. It also improves understanding of FPGA interfacing and digital system design.

Table of Contents

Introduction	6
OBJECTIVES.....	Error! Bookmark not defined.
Background.....	7
Understanding FPGA: Field-Programmable Gate Array	7
Introduction to Xilinx	9
Sample Verilog CODE	11
Tools / Environment Used.....	13
Design Methodology	14
[Put your design methodology here]	Error! Bookmark not defined.
Verilog Code	14
Simulation and Verification.....	17
Implementation on Spartan 7 FPGA.....	19
Spartan 7 FPGA development board	19
FPGA Configuration.....	20
Testing and Results.....	22
Conclusion	23
References	24

Introduction

This project implements a 4-bit binary-coded hexadecimal decoder using Verilog HDL, targeting a Spartan-7 FPGA. The design reads 4-bit binary input via switches and displays the corresponding hexadecimal digit on a 7-segment display. It supports multiple display selection, reset functionality, and enables digital visualization of binary input on hardware.

Objectives

- ☐ To design a 4-bit binary-coded hexadecimal decoder using Verilog HDL.
- ☐ To implement the design on a **Spartan-7 FPGA** development board.
- ☐ To display the decoded hexadecimal value on a **7-segment display**.
- ☐ To provide **real-time interaction** using input switches and a control button.
- ☐ To enhance practical understanding of **digital logic design** and **FPGA interfacing**.
- ☐ To learn and apply **constraint mapping, synthesis, and simulation** using Vivado.
- ☐ To create a user-selectable display for flexible output visualization.
- ☐ To learn about how to convert Binary => Onehot => 7-segment.

Background

Understanding FPGA: Field-Programmable Gate Array

Field-Programmable Gate Arrays (FPGAs) are integrated circuits designed to be configured by the customer or designer after manufacturing—hence "field-programmable". They consist of an array of programmable logic blocks and a hierarchy of reconfigurable interconnects that allow these blocks to be wired together. This structure allows FPGAs to be programmed to perform complex digital computations and tasks.

Key Characteristics of FPGAs

1. **Reconfigurability:-** FPGAs can be reprogrammed to perform different functions or to optimize the performance of existing functions, making them highly versatile for various applications.
2. **Parallel Processing:-** Unlike traditional processors that execute instructions sequentially, FPGAs can perform multiple operations in parallel, significantly increasing computational efficiency for specific tasks.
3. **Low Latency:-** FPGAs can process data with very low latency because they can be tailored to the specific needs of an application, reducing the need for additional processing steps.
4. **Customizability:-** FPGAs allow for the implementation of custom hardware designs, which can be optimized for performance, power consumption, or both.

FPGA Architecture

1. **Configurable Logic Blocks (CLBs):-** CLBs are the primary logic units in an FPGA. They contain look-up tables (LUTs), flip-flops, and multiplexers that can be configured to implement various logic functions.
2. **Interconnects:-** FPGAs use a complex matrix of interconnects to route signals between CLBs. These interconnects can be dynamically programmed to connect the desired logic blocks.
3. **I/O Blocks:-** Input/ Output blocks interface the FPGA with external devices. These blocks support various I/O standards and protocols, enabling the FPGA to communicate with other components in a system.
4. **Memory Blocks:-** FPGAs include dedicated memory blocks for temporary data storage, which can be used for tasks like buffering and caching.
5. **Clock Management:-** Clock management resources, such as phase-locked loops (PLLs) and clock dividers, are included to manage the timing of operations within the FPGA.

FPGA Development Process

1. **Design Entry:-** The design process starts with defining the logic and functionality, typically using Hardware Description Languages (HDLs) like VHDL or Verilog.
2. **Simulation:-** Before implementing the design on hardware, it is simulated to verify its correctness. Simulation tools allow designers to test and debug their logic.

3. **Synthesis:-** The HDL code is synthesized into a netlist, which describes the hardware components and their interconnections.
4. **Implementation:-** The netlist is mapped to the FPGA's resources (CLBs, interconnects, etc.). This step involves place-and-route algorithms to optimize the design's performance.
5. **Programming:-** The final step is to generate a bitstream file, which is used to program the FPGA. This file configures the FPGA to implement the desired logic.
6. **Verification and Testing:-** After programming the FPGA, the design is verified and tested in hardware to ensure it meets all functional and performance requirements.

Applications of FPGAs

1. **Telecommunications:-** FPGAs are used in network routers, switches, and base stations to perform high-speed data processing and protocol management.
2. **Consumer Electronics:-** In devices like digital cameras, smartphones, and gaming consoles, FPGAs are used for image processing, signal processing, and interfacing with other components.
3. **Automotive:-** FPGAs are employed in advanced driver-assistance systems (ADAS), infotainment systems, and electronic control units (ECUs).
4. **Industrial Automation:-** In industrial control systems, FPGAs provide the flexibility and performance required for real-time processing and control.
5. **Aerospace and Defense:-** FPGAs are used in radar systems, communication devices, and avionics due to their reliability and reconfigurability.
6. **Medical Devices:-** FPGAs enable high-performance processing in medical imaging systems, diagnostic equipment, and portable medical devices.

Advantages of FPGAs

- **Flexibility:-** Ability to reprogram for different applications and updates.
- **Performance** High-speed data processing with parallel execution.
- **Customization:** Tailored hardware solutions for specific needs.
- **Scalability:** Suitable for a wide range of applications from small devices to large systems.

Disadvantages of FPGAs

- **Cost:** Higher initial cost compared to ASICs for large-volume production.
- **Power Consumption:** Can consume more power than fixed-function devices.
- **Complexity:** Requires specialized knowledge for design and programming.

Introduction to Verilog Programming

Verilog is a Hardware Description Language (HDL) used to model digital electronic systems. It enables designers to write code that describes the structure, behavior, and functionality of digital circuits like gates, counters, multiplexers, and CPUs.

Verilog supports both structural modeling (connecting components) and behavioral modeling (using logical operations and algorithms), making it suitable for both low-level gate designs

and high-level system simulations. It is widely used for FPGA and ASIC development and is supported by industry-standard tools such as Xilinx Vivado, ModelSim, and Synopsys VCS.

Using Verilog, developers can simulate their designs before implementation, helping to detect logic errors early in the development cycle. This makes Verilog essential for modern digital design workflows, ensuring accurate and efficient hardware development.

Introduction to VHDL Programming

VHDL (VHSIC (Very High Speed Integrated Circuits) hardware description language) is commonly used as a design-entry language for field-programmable gate arrays and application-specific integrated circuits in electronic design automation of digital circuits. VHDL was originally developed at the behest of the US Department of Defense in order to document the behavior of the ASICs that supplier companies were including in equipment. That is to say, VHDL was developed as an alternative to huge, complex manuals which were subject to implementation-specific details.

The idea of being able to simulate this documentation was so obviously attractive that logic simulators were developed that could read the VHDL files. The next step was the development of logic synthesis tools that read the VHDL, and output a definition of the physical implementation of the circuit. Modern synthesis tools can extract RAM, counter, and arithmetic blocks out of the code, and implement them according to what the user specifies. Thus, the same VHDL code could be synthesized differently for lowest area, lowest power consumption, highest clock speed, or other requirements.

VHDL borrows heavily from the Ada programming language in both concepts (for example, the slice notation for indexing part of a one-dimensional array) and syntax. VHDL is not a case sensitive language. One can design hardware in a VHDL IDE (such as Xilinx or Quartus) to produce the RTL schematic of the desired circuit. After that, the generated schematic can be verified using simulation software (such as ModelSim) which shows the waveforms of inputs and outputs of the circuit after generating the appropriate test bench. To generate an appropriate test bench for a particular circuit or VHDL code, the inputs have to be defined correctly. For example, for clock input, a loop process or an iterative statement is required.

The key advantage of VHDL when used for systems design is that it allows the behavior of the required system to be described (modeled) and verified (simulated) before synthesis tools translate the design into real hardware (gates and wires). Another benefit is that VHDL allows the description of a concurrent system (many parts, each with its own sub-behavior, working together at the same time). VHDL is a Dataflow language, unlike procedural computing languages such as BASIC, C, and assembly code, which all run sequentially, one instruction at a time.

Introduction to Xilinx

Xilinx (an Electronics Company) is the world's largest supplier of programmable logic devices and the inventor of the field programmable gate array (FPGA). Today Xilinx is the worldwide leader of complete programmable logic solutions.

Xilinx has two main FPGA families – the high-performance video Virtex series and the high-volume Spartan series. Each model series has been released in multiple generations since its launch. The latest Virtex-6 and Spartan-6 FPGA families are said to consume 50 % less power , cost 20 % less , and have up to twice the logic capacity of previous generations of FPGAs. The Spartan series targets applications with low power footprint, extremes cost sensitivity and high volume e.g. displays, set top boxes wireless routers and other applications.

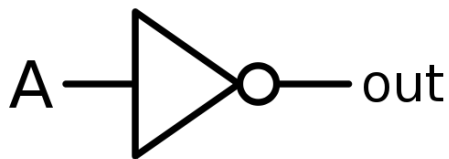
The ISE Design Suite is the central electronic design automation (EDA) product family sold by Xilinx .The ISE Design Suite features include design entry and synthesis supporting Verilog or VHDL ,place and route (PAR), completed verification and Chipscope pro tools , and creation of the bit files that are used to configure the chip .

Xilinx designs, develops and markets programmable logic products including integrated circuits (ICs), software design tools and services . Xilinx sells both FPGAs and CPLDs (Complex Programmable Logic Devices) for electronic equipment manufacturers.

In this project we used the Xilinx Vivado 2023.2 version to complete our project.

Sample Verilog CODE

1-BIT NOT GATE



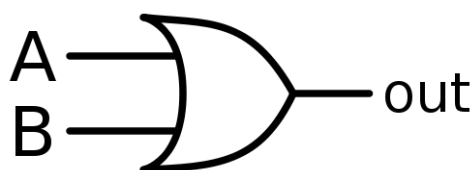
VERILOG CODE

```
module not_gate (  
    input wire a,    // 1-bit input  
    output wire out  // 1-bit output  
);  
  
assign y = ~a;      // NOT operation  
  
endmodule
```

TEST BENCH WAVEFORM



1-BIT OR GATE



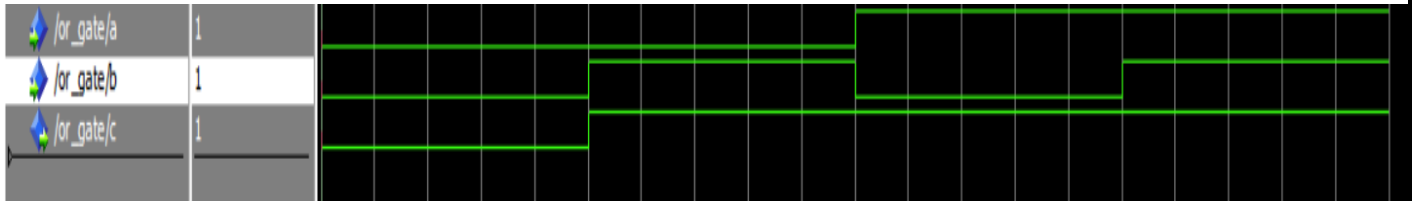
VERILOG CODE

```
module OR_gate(  
    input wire a, b, // 1-bit input  
    output wire out);  
  
assign out = a | b;  
  
endmodule
```

TEST BENCH WAVEFORM

TRUTH TABLE

Inputs		Output
A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1



4-BIT OR GATE

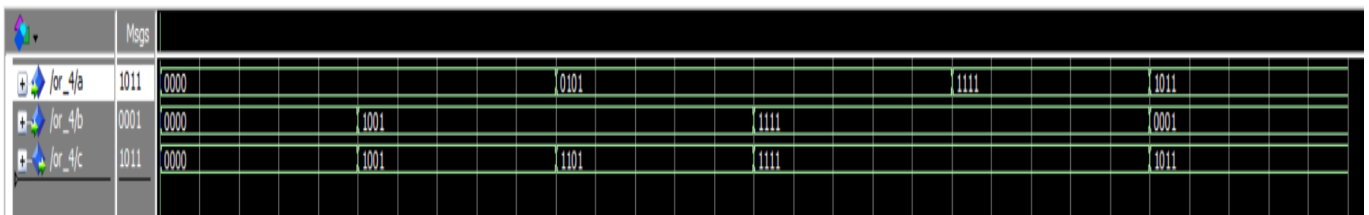
VERILOG CODE

```
module or4 (
    input wire [3:0] a, // 4-bit input
    output wire y      // 1-bit output
);

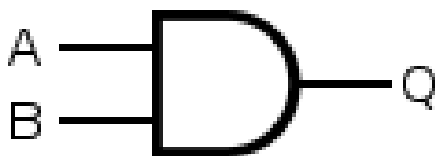
    assign y = a[0] | a[1] | a[2] | a[3]; // OR all 4 bits
```

endmodule

TEST BENCH WAVEFORM



1-BIT AND GATE



TRUTH TABLE

Inputs		Output
A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

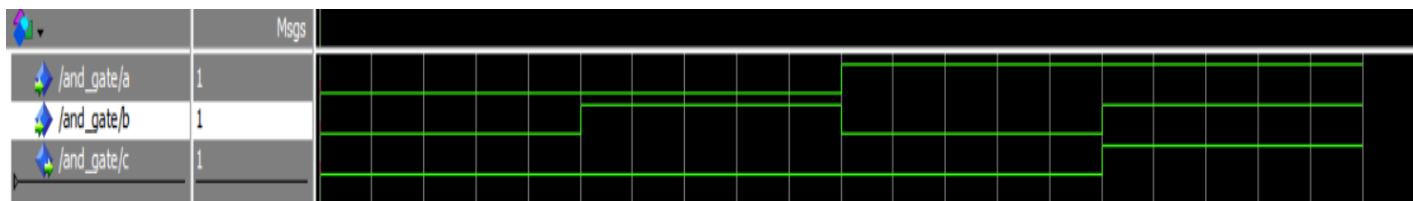
VERILOG CODE

```
module and (
    input wire a, // 1-bit input
    input wire b, // 1-bit input
    output wire y // 1-bit output
);

    assign y = a & b; // AND operation
```

endmodule

TEST BENCH WAVEFORM



4-BIT AND GATE

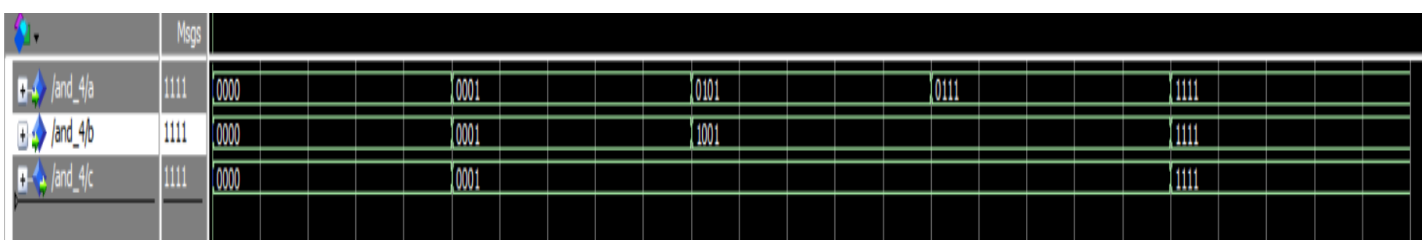
VERILOG CODE

```
module and4bit (
    input wire [3:0] a, // 4-bit input A
    input wire [3:0] b, // 4-bit input B
    output wire [3:0] y // 4-bit output Y = A & B
);

    assign y = a & b;      // Bitwise AND operation

endmodule
```

TEST BENCH WAVEFORM



Add more sample code and testbench waveform from your classwork & assignments

Tools / Environment Used

HARDWARES:

1. PINE S7 FPGA DEVELOPMENT BOARD : SPARTAN 7
2. Hard disk : 320 GB or higher
3. Processor : Intel i3
4. Ram : 4 GB and above
5. USB CABLE : B TYPE

SOFTWARES:

- | | |
|-------------------------|---------------------------------|
| 1. OPERATING SYSTEM | : Windows 10/11 |
| 2. DEVELOPMENT SOFTWARE | : Xilinx VIVADO 2023.2 or above |

Design Methodology

The design of the 4-bit Binary Coded Hexadecimal to 7-Segment Display Decoder on Spartan-7 FPGA followed a systematic hardware development process:

1. Requirement Analysis

- Identified input (4-bit binary) and output (7-segment display) behavior.
- Defined display selection mechanism for multiple digits.

2. Behavioral Modeling (Verilog HDL)

- Developed the logic for binary-to-hexadecimal conversion.
- Encoded 7-segment display values for hexadecimal digits (0–F).
- Integrated optional display selection using anode control.

3. Functional Simulation

- Wrote a testbench to verify output segments against all 4-bit inputs.
- Debugged using waveform analysis in Vivado's simulator.

4. Synthesis & Implementation

- Used Xilinx Vivado to synthesize and implement the design on the Spartan-7 FPGA.

5. Constraints Mapping

- Assigned FPGA pins using an XDC file based on board's switch, segment, and anode connections.

6. Bitstream Generation & Testing

- Generated bitstream and programmed the board.
- Validated real-time behavior using physical inputs and display output.

Verilog Code

top.v

```
`timescale 1ns / 1ps
module top (
    input wire [3:0] sw,    // 4-bit binary input from switches
```

```

    input wire    btn,    // Enable button
    output wire [6:0] seg,    // 7-segment segments
    output wire [3:0] an,    // Anode (enable digit 0)
    output wire [3:0] hex    // Hex digit output as 4-bit binary
);
    wire [15:0] onehot_out;
    wire [3:0] hex_internal;
    assign an = 4'b1110; // Activate only digit 0
    assign hex = hex_internal;
    binary_to_onehot decoder (
        .enable(btn),
        .bin_in(sw),
        .hex_out(onehot_out)
    );
    onehot_to_7seg display (
        .onehot(onehot_out),
        .seg(seg),
        .hex_digit(hex_internal)
    );
endmodule

```

binary_to_onehot.v

```

module binary_to_onehot (
    input wire    enable,
    input wire [3:0] bin_in,
    output reg [15:0] hex_out
);
    always @(*) begin
        if (enable)
            hex_out = 16'b0000000000000001 << bin_in;
        else

```

```

        hex_out = 16'b0000000000000000;

    end

endmodule

onehot_to_7seg.v

module onehot_to_7seg (
    input wire [15:0] onehot,
    output reg [6:0] seg,
    output reg [3:0] hex_digit
);

always @(*) begin
    case (onehot)
        16'b0000000000000001: begin seg = 7'b1000000; hex_digit = 4'h0; end // 0
        16'b0000000000000010: begin seg = 7'b1111001; hex_digit = 4'h1; end // 1
        16'b0000000000000100: begin seg = 7'b0100100; hex_digit = 4'h2; end // 2
        16'b0000000000001000: begin seg = 7'b0110000; hex_digit = 4'h3; end // 3
        16'b0000000000010000: begin seg = 7'b0011001; hex_digit = 4'h4; end // 4
        16'b0000000000100000: begin seg = 7'b0010010; hex_digit = 4'h5; end // 5
        16'b0000000001000000: begin seg = 7'b0000010; hex_digit = 4'h6; end // 6
        16'b0000000010000000: begin seg = 7'b1111000; hex_digit = 4'h7; end // 7
        16'b0000000100000000: begin seg = 7'b0000000; hex_digit = 4'h8; end // 8
        16'b0000001000000000: begin seg = 7'b0010000; hex_digit = 4'h9; end // 9
        16'b0000010000000000: begin seg = 7'b0001000; hex_digit = 4'hA; end // A
        16'b0000100000000000: begin seg = 7'b0000011; hex_digit = 4'hB; end // B
        16'b0001000000000000: begin seg = 7'b1000110; hex_digit = 4'hC; end // C
        16'b0010000000000000: begin seg = 7'b0100001; hex_digit = 4'hD; end // D
        16'b0100000000000000: begin seg = 7'b0000110; hex_digit = 4'hE; end // E
        16'b1000000000000000: begin seg = 7'b0001110; hex_digit = 4'hF; end // F
        default: begin seg = 7'b1111111; hex_digit = 4'hF; end // blank
    endcase
end

endmodule

```

Simulation and Verification

The Verilog code is first simulated using simulation tool such of Vivado to verify the correctness of the logic. The simulation involves applying different inputs and observing the corresponding outputs on the Spartan 7 FPGA Development Board.

Verilog TESTBENCH FILE CODE

```
`timescale 1ns / 1ps

module top_tb;

    reg [3:0] sw;
    reg      btn;
    wire [6:0] seg;
    wire [3:0] an;
    wire [3:0] hex;

    top uut (
        .sw(sw),
        .btn(btn),
        .seg(seg),
        .an(an),
        .hex(hex)
    );

    initial begin
        btn = 0;
        sw = 4'b0000;
        #10 btn = 1;

        // Step through all hex digits
        for (integer i = 0; i < 16; i = i + 1) begin
            sw = i;
            #10;
        end
    end
end
```

```

// Disable output

btn = 0;

#10;

$finish;

end

initial begin

    $monitor("Time=%0t | sw(bin)=%b | hex_out=0x%h | seg=%b | an=%b", $time, sw, hex,
seg, an);

    end

endmodule

```

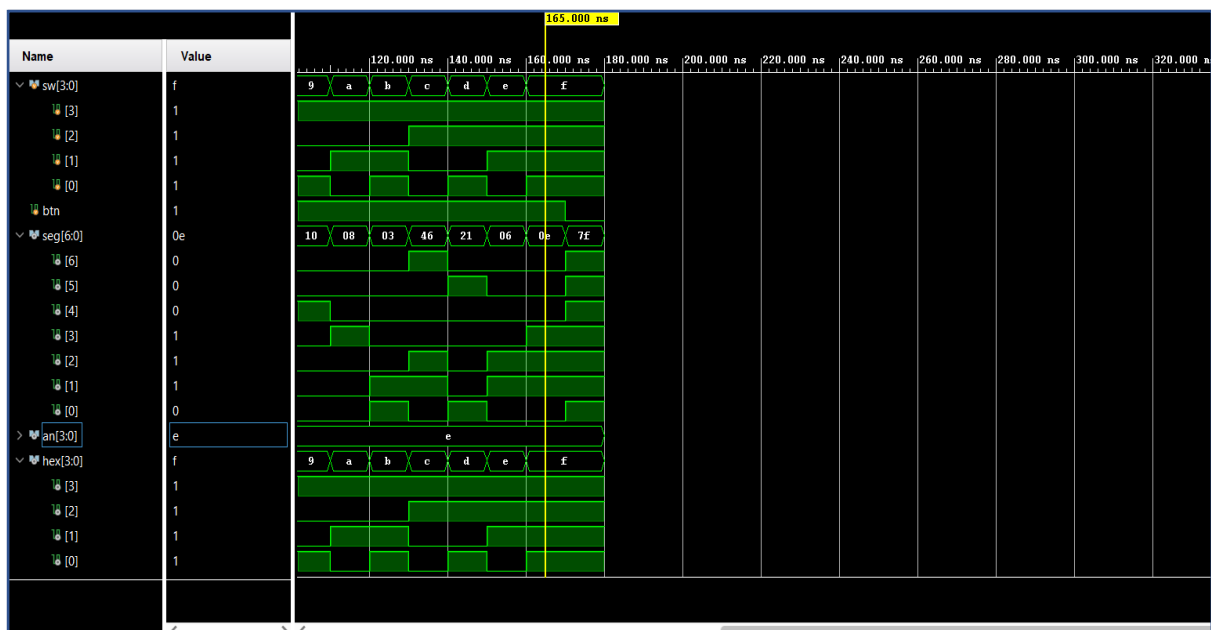
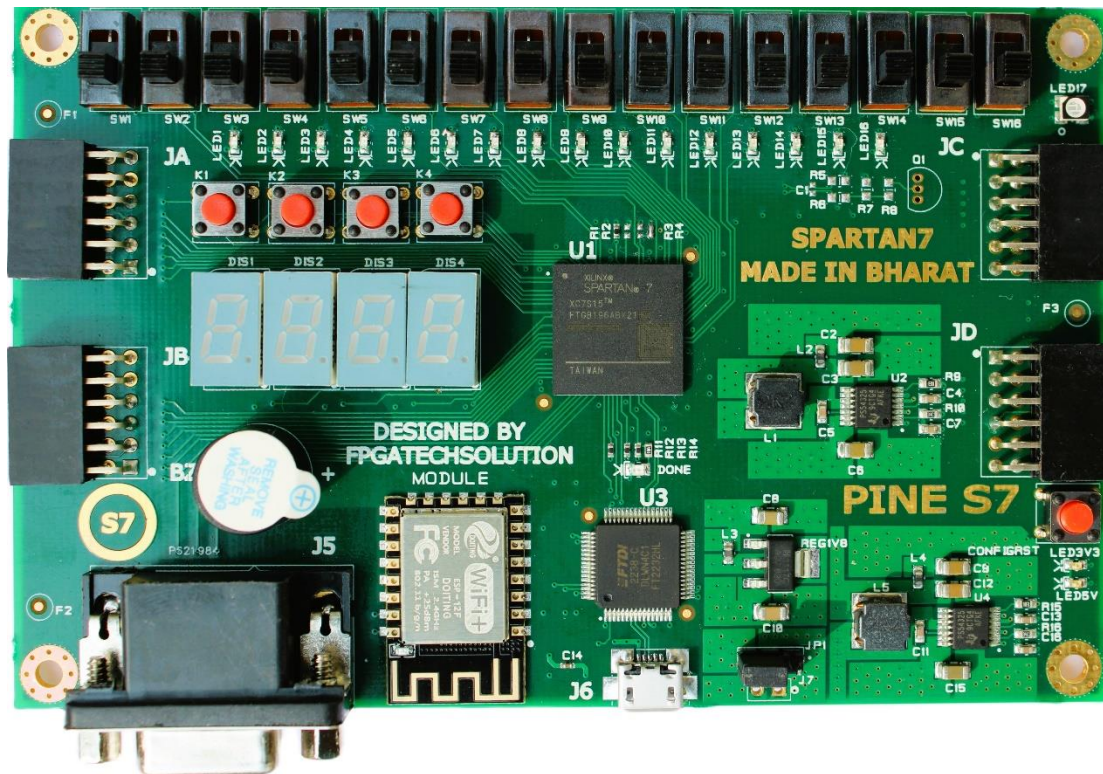


Figure: Test Bench Wave Form

Implementation on Spartan 7 FPGA

Spartan 7 FPGA development board



PINE S7 is an easy to use FPGA Development board featuring SPARTAN7 FPGA (XC7S15FTGB196-1), with FTDI's FT232H Dual-Channel USB device, which support Xilinx Vivado, SDK & Vitis to program FPGA. It is specially designed for the development and integration of FPGA based accelerated features to other designs. This board works with USB power however provision is given for external 5V power supply

Features

- 16 user switches
- 16 user LEDs
- 4 user pushbuttons
- 100MHz clock
- 4-digit 7-segment display
- Four Pmod ports
- ESP-12E WiFi
- Buzzer
- RGB LED
- VGA output
- USB-UART Bridge
- Serial Flash
- USB-JTAG port for FPGA programming and communication (support Vivado)

FPGA Configuration

The synthesized design is implemented on the FPGA using Vivado Design Suite. The seven-segment display is connected to the appropriate GPIO pins of the Spartan 7 FPGA development board. Pin assignments are made according to the board's pinout configuration.

Constraint File (.XDC file)

#Slide Switches (as inputs)

```
set_property -dict { PACKAGE_PIN A5  IOSTANDARD LVCMOS33 } [get_ports {  
SW[0] }];
```

```
set_property -dict { PACKAGE_PIN B6  IOSTANDARD LVCMOS33 } [get_ports {  
SW[1] }];
```

```
set_property -dict { PACKAGE_PIN F4  IOSTANDARD LVCMOS33 } [get_ports {  
SW[2] }];
```

```
set_property -dict { PACKAGE_PIN C3  IOSTANDARD LVCMOS33 } [get_ports {  
SW[3] }];
```

##seven segment

```
set_property -dict { PACKAGE_PIN G4  IOSTANDARD LVCMOS33 } [get_ports {  
seg[0] }];
```

```
set_property -dict { PACKAGE_PIN H3  IOSTANDARD LVCMOS33 } [get_ports {  
seg[1] }];
```

```
set_property -dict { PACKAGE_PIN J3  IOSTANDARD LVCMOS33 } [get_ports { seg[2]  
}];
```

```
set_property -dict { PACKAGE_PIN L3  IOSTANDARD LVCMOS33 } [get_ports {  
seg[3] }];
```

```
set_property -dict { PACKAGE_PIN K3  IOSTANDARD LVCMOS33 } [get_ports {  
seg[4] }];
```

```
set_property -dict { PACKAGE_PIN H4  IOSTANDARD LVCMOS33 } [get_ports {  
seg[5] }];
```

```
set_property -dict { PACKAGE_PIN J4  IOSTANDARD LVCMOS33 } [get_ports { seg[6]  
}];
```

an (display selector)

```
set_property -dict { PACKAGE_PIN H1  IOSTANDARD LVCMOS33 } [get_ports { an[0]
}];
```

```
set_property -dict { PACKAGE_PIN H2  IOSTANDARD LVCMOS33 } [get_ports { an[1]
}];
```

```
set_property -dict { PACKAGE_PIN F1  IOSTANDARD LVCMOS33 } [get_ports { an[2]
}];
```

```
set_property -dict { PACKAGE_PIN G1  IOSTANDARD LVCMOS33 } [get_ports { an[3]
}];
```

Enable button(button)

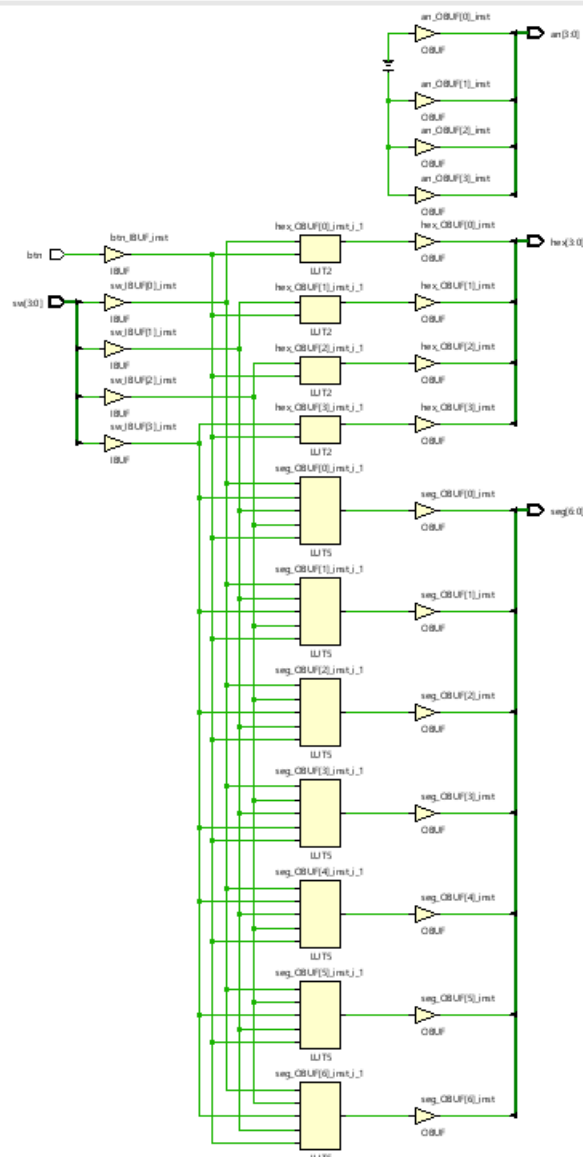
```
set_property -dict { PACKAGE_PIN B5  IOSTANDARD LVCMOS33 } [get_ports { btn
}];
```

Testing and Results

The FPGA is configured with the synthesized design, and various sw inputs are applied to verify the numerical representation on the seven-segment display. The testing process involves:

- Configuring the FPGA with the synthesized design.

31 Cells 20 I/O Ports 38 Nets



Synthesis Schematics

- Applying test inputs using switches or buttons on the FPGA board.
- Observing and verifying the outputs on the seven-segment display.

```

Time=0 | sw(bin)=0000 | hex_out=0xf | seg=1111111 | an=1110
Time=10000 | sw(bin)=0000 | hex_out=0x0 | seg=1000000 | an=1110
Time=20000 | sw(bin)=0001 | hex_out=0x1 | seg=1111001 | an=1110
Time=30000 | sw(bin)=0010 | hex_out=0x2 | seg=0100100 | an=1110
Time=40000 | sw(bin)=0011 | hex_out=0x3 | seg=0110000 | an=1110
Time=50000 | sw(bin)=0100 | hex_out=0x4 | seg=0011001 | an=1110
Time=60000 | sw(bin)=0101 | hex_out=0x5 | seg=0010010 | an=1110
Time=70000 | sw(bin)=0110 | hex_out=0x6 | seg=0000010 | an=1110
Time=80000 | sw(bin)=0111 | hex_out=0x7 | seg=1111000 | an=1110
Time=90000 | sw(bin)=1000 | hex_out=0x8 | seg=0000000 | an=1110
Time=100000 | sw(bin)=1001 | hex_out=0x9 | seg=0010000 | an=1110
Time=110000 | sw(bin)=1010 | hex_out=0xa | seg=0001000 | an=1110
Time=120000 | sw(bin)=1011 | hex_out=0xb | seg=0000011 | an=1110
Time=130000 | sw(bin)=1100 | hex_out=0xc | seg=1000110 | an=1110
Time=140000 | sw(bin)=1101 | hex_out=0xd | seg=0100001 | an=1110
Time=150000 | sw(bin)=1110 | hex_out=0xe | seg=0000110 | an=1110
Time=160000 | sw(bin)=1111 | hex_out=0xf | seg=0001110 | an=1110
Time=170000 | sw(bin)=1111 | hex_out=0xf | seg=1111111 | an=1110

```

TCL console display

Conclusion

FPGAs are powerful tools in the field of digital design, offering unparalleled flexibility and performance for a wide range of applications. Their ability to be reprogrammed and customized makes them ideal for industries where rapid prototyping and iteration are crucial. As technology advances, FPGAs continue to play a vital role in the development of modern electronic systems.

The project “4-bit Binary Coded Hexadecimal to 7-Segment Display Decoder on Spartan-7 FPGA” was successfully designed, simulated, and synthesized using Verilog. It efficiently converts 4-bit binary inputs into hexadecimal outputs displayed on a 7-segment display. The implementation enhances understanding of digital logic, FPGA interfacing, and real-time hardware display control using Spartan-7.

References

- Xilinx Inc., *Spartan-7 FPGA Family Data Sheet*, <https://www.xilinx.com>.
- Brown, S., & Vranesic, Z., *Fundamentals of Digital Logic with Verilog Design*, McGraw-Hill Education, 3rd Edition.
- Harris, D., & Harris, S., *Digital Design and Computer Architecture*, Morgan Kaufmann, 2nd Edition.
- Verilog HDL Reference Guide – Synopsys Inc.
- Digilent Inc., Basys-3 and Nexys-A7 *Board Reference Manual*, <https://reference.digilentinc.com>.
- TutorialsPoint. *Verilog - Basic Concepts*. <https://www.tutorialspoint.com/verilog/index.htm>
- All About Circuits – *7-Segment Displays*, <https://www.allaboutcircuits.com>
- FPGA4Student – *Verilog HDL Examples and Projects*, <https://www.fpga4student.com>

THANK YOU